

PLANDEX_INDEX

Plandex Exploration - Complete Documentation Index

This directory contains a comprehensive analysis of Plandex, an AI coding agent designed for large-scale development tasks. All documents are optimized for actionable porting to HelixCode.

Documents Overview

1. PLANDEX_ANALYSIS.md (592 lines, 21 KB)

Comprehensive Technical Analysis

The primary reference document containing: - 8 major sections with complete feature breakdown - All supported LLM providers (12 total) with authentication patterns - Supported models (25+) with variants and packs (16 total) - Key technical implementations (architecture, database, file editing, context management) - 9 unique features that differentiate Plandex - 10 key technical patterns for porting - Database schema and infrastructure details - Porting recommendations with effort estimates

Best for: Understanding the complete system, making architectural decisions, identifying features to port

Key Insights: - 2M token effective context window with smart filtering - 5-level autonomy system (none → full automation) - Role-based model routing (9 specialized roles) - Cumulative diff review sandbox (unique differentiator) - Multi-provider fallback chain with automatic failover

2. PLANDEX_QUICK_REFERENCE.md (342 lines, 12 KB)

Visual Guide & Quick Lookup

Fast reference guide with: - Architecture diagram (ASCII art) - Provider fallback chain visualization - Core features matrix (8 features × implementation) - Model roles flow diagram - Autonomy levels hierarchy - Database schema essentials - File edit fallback chain - Context management pipeline - Configuration hierarchy -

Model pack overview (16 packs) - Critical implementation files (LOC count) -
Provider comparison table - Environment variables reference

Best for: Quick lookups during development, team onboarding, architecture discussions

Use Cases: - “Which providers does Plandex support?” → Check Provider Summary - “How does context work?” → See Context Management Pipeline - “What’s the fallback chain?” → View Provider Fallback Chain - “How many autonomy levels?” → Check Autonomy Levels section

3. PLANDEX_PORTING_CHECKLIST.md (512 lines, 17 KB)

Detailed Implementation Roadmap

Step-by-step checklist with: - **Phase 1** (2-3 weeks): LLM Provider Integration - 12 provider implementations - Credential management system - Fallback chain architecture - LiteLLM proxy setup

- **Phase 2** (2-3 weeks): Model Configuration & Role System
 - Database schema for models
 - 9 role definitions
 - 25+ model definitions
 - 16 model pack configurations
 - Model resolution engine
- **Phase 3** (2-3 weeks): Context Management System
 - Project map generation (tree-sitter)
 - 4-stage context loading pipeline
 - Smart context filtering
 - Prompt caching integration
- **Phase 4** (1-2 weeks): File Editing & Diff System
 - Diff generation (git-based)
 - Structured AST-aware edits
 - 5-level fallback chain
 - Change application and staging
- **Phase 5** (1-2 weeks): Plan Management & Versioning
 - Plan CRUD operations
 - Full versioning system
 - Branching support
 - Conversation management
- **Phase 6** (1-2 weeks): Autonomy System
 - 5 preset levels + individual flags
 - Runtime autonomy checks
 - Safety features

- **Phase 7** (1-2 weeks): Execution & Debugging
 - Command execution
 - Auto-debugging (terminal + browser)
 - Rollback on failure
- **Phase 8** (1 week): Git Integration
 - Git operations
 - Commit message generation
 - Optional auto-commit
- **Phase 9** (2-3 weeks): Testing & Validation
 - Unit tests
 - Integration tests
 - E2E tests
 - Load testing
- **Phase 10** (1 week): Documentation & Finalization
 - Code documentation
 - User guides
 - Developer guides
 - Migration guides

Implementation Details: - Specific Go package structure - Database migrations - Handler endpoints (/api/v1/...) - Exact config structures (with Go code examples) - Reusable HelixCode components - External dependencies to add - Success criteria (11 checkboxes)

Timeline: 8-12 weeks for MVP

Best for: Day-to-day development, task assignment, progress tracking, sprint planning

Use Cases: - “What should we implement first?” → Start Phase 1 - “How do we structure LLM providers?” → See Phase 1.1 - “What handlers do we need?” → Check Phase 2.6, 3.4, 4.4, 5.6 - “Are we ready to ship?” → Check Success Criteria

How to Use These Documents

For Project Leads

1. Start with **PLANDEX_ANALYSIS.md** sections 1-2 (Features & Providers)
2. Review **PLANDEX_QUICK_REFERENCE.md** Architecture section
3. Use **PLANDEX_PORTING_CHECKLIST.md** for timeline and resource planning
4. Reference “Porting Recommendations” in **PLANDEX_ANALYSIS.md** (Section 8)

For Architects

1. Read **PLANDEX_ANALYSIS.md** Section 4 (Key Technical Implementations)
2. Study **PLANDEX_QUICK_REFERENCE.md** Diagrams (Architecture, Fallback Chain, Context Pipeline)
3. Review **PLANDEX_PORTING_CHECKLIST.md** Phases 1-2 (Infrastructure setup)
4. Check Section 6 of **ANALYSIS.md** for technical patterns

For Developers

1. Use **PLANDEX_QUICK_REFERENCE.md** as daily lookup
2. Follow **PLANDEX_PORTING_CHECKLIST.md** for your phase
3. Refer to **PLANDEX_ANALYSIS.md** Section 4 when implementing
4. Check section 4.2+ for specific pattern details

For DevOps/Infrastructure

1. Review **PLANDEX_QUICK_REFERENCE.md** “Database Schema” and “Environment Variables”
2. Check **PLANDEX_ANALYSIS.md** Section 7 (Database & Infrastructure)
3. Follow Phase 1.4 in **PORTING_CHECKLIST.md** for LiteLLM Proxy
4. Reference Docker/container setup notes

For QA/Testing

1. Review **PLANDEX_PORTING_CHECKLIST.md** Phase 9 (Testing)
 2. Check **PLANDEX_ANALYSIS.md** Section 5.8 (Browser Debugging)
 3. Use **QUICK_REFERENCE.md** to understand workflows
 4. Reference autonomy levels and edge cases from **ANALYSIS.md**
-

Quick Navigation by Topic

LLM Providers

- Quick list: **QUICK_REFERENCE.md** → Provider Summary table
- Detailed: **ANALYSIS.md** → Section 2 (Supported LLM Providers)
- Implementation: **CHECKLIST.md** → Phase 1 (Provider Integration)
- Credentials: **QUICK_REFERENCE.md** → Environment Variables

Models & Model Packs

- Overview: **ANALYSIS.md** → Section 3

- Visual: QUICK_REFERENCE.md → Model Packs Overview
- Database: CHECKLIST.md → Phase 2.1-2.4
- Configuration: ANALYSIS.md → Section 3.4

Context Management

- Concepts: ANALYSIS.md → Section 1.3
- Pipeline: QUICK_REFERENCE.md → Context Management Pipeline diagram
- Implementation: CHECKLIST.md → Phase 3
- Caching: ANALYSIS.md → Section 4.4

Plan & Versioning

- Features: ANALYSIS.md → Sections 1.1, 1.7
- Implementation: CHECKLIST.md → Phase 5
- Git integration: ANALYSIS.md → Section 4.7, CHECKLIST.md → Phase 8

Autonomy System

- Overview: ANALYSIS.md → Section 1.5
- Visual: QUICK_REFERENCE.md → Autonomy Levels diagram
- Implementation: CHECKLIST.md → Phase 6

File Editing & Diffs

- Features: ANALYSIS.md → Section 1.4
- Technical: ANALYSIS.md → Section 4.3
- Fallback chain: QUICK_REFERENCE.md → File Edit Fallback Chain diagram
- Implementation: CHECKLIST.md → Phase 4

Execution & Debugging

- Features: ANALYSIS.md → Section 1.6
- Browser debugging: ANALYSIS.md → Section 5.8
- Implementation: CHECKLIST.md → Phase 7

Key Metrics at a Glance

Metric	Value	Reference
Total Providers	12	ANALYSIS.md 2.1
Models	25+	ANALYSIS.md 3.2
Model Packs	16	ANALYSIS.md 3.4

Metric	Value	Reference
Model Roles	9	ANALYSIS.md 1.8
Autonomy Levels	5	ANALYSIS.md 1.5
Database LOC	8,351	ANALYSIS.md 4.2
Context Window	2M tokens	ANALYSIS.md 1
Project Map Size	20M+ tokens	ANALYSIS.md 3.2
Estimated Porting	8-12 weeks	CHECKLIST.md Intro
File Edit Fallback Stages	5	QUICK_REFERENCE.md

Key Differentiators to Understand

- Cumulative Diff Review Sandbox** (ANALYSIS.md 5.1)
 - Most similar tools apply changes immediately
 - Plandex keeps all changes in sandbox until approved
 - Enables selective file rejection and complete rollback
 - Smart Context Window** (ANALYSIS.md 5.4)
 - Dynamically resizes per task step
 - Solves “n-file problem” (doesn’t reload all 10 when editing 1)
 - Only loads files relevant to current step
 - Progressive Autonomy** (ANALYSIS.md 5.2)
 - 5 preset levels from manual to full automation
 - Each level has individual flag overrides
 - Reconfigurable at runtime
 - Multi-role Model System** (ANALYSIS.md 5.3)
 - Different specialized models for planning, coding, building
 - Enables mixing models without explicit chains
 - Falls back gracefully if role not configured
 - Provider Fallback Chain** (ANALYSIS.md 2.2)
 - Automatic retry with alternative provider on error
 - Priority order: Direct → Aggregator → Error
 - Reduces need for extensive provider configuration
-

Common Questions Answered

Q: How do I add a new LLM provider? A: See CHECKLIST.md Phase 1 - structure `internal/llm/provider/` and implement Provider interface

Q: What’s the difference between a model pack and a model? A: Model = individual LLM, Pack = role → model mapping. See ANALYSIS.md 3.4

Q: How does context caching work? A: See QUICK_REFERENCE.md Context Caching section and ANALYSIS.md 4.4

Q: What happens if a model fails? A: See QUICK_REFERENCE.md Provider Fallback Chain - automatic retry with next provider

Q: Can we customize autonomy levels? A: Yes, see CHECKLIST.md Phase 6 - preset levels + individual flag overrides

Q: How does the file editing fallback chain work? A: See QUICK_REFERENCE.md File Edit Fallback Chain - 5 stages from targeted to manual fix

Q: What database tables do we need? A: See QUICK_REFERENCE.md Database Schema (Essentials) and CHECKLIST.md Phase 2.1, 3.1, 5.1

Q: How long will porting take? A: 8-12 weeks for MVP. See CHECKLIST.md timeline overview

Document Cross-References

PLANDEX_ANALYSIS.md

- └ Explains WHAT and WHY
- └ Referenced by: QUICK_REFERENCE.md (visual), CHECKLIST.md (HOW)
- └ Sections 1-2: Best for features, Section 4-6: Best for architecture

PLANDEX_QUICK_REFERENCE.md

- └ Quick lookup and visual learning
- └ Complements: ANALYSIS.md with diagrams
- └ Best for: Team alignment, daily development reference

PLANDEX_PORTING_CHECKLIST.md

- └ Explains HOW to implement
 - └ Implements: ANALYSIS.md Section 4-6 patterns
 - └ Includes: Code structure, handlers, database schema
-

Maintenance Notes

These documents were generated from Plandex v2.x codebase. Key files analyzed:
- app/shared/ai_models_*.go (Models and providers) - app/server/db/*.go (Database schema - 8,351 LOC) - app/server/model/plan/ (Execution logic) - app/server/syntax/ (File editing) - app/cli/ (CLI implementation) - docs/docs/ (User documentation)

Next Steps

1. **Review:** Read PLANDEX_ANALYSIS.md Section 1 (Core Features)
 2. **Visualize:** Study PLANDEX_QUICK_REFERENCE.md Architecture diagrams
 3. **Plan:** Use PLANDEX_PORTING_CHECKLIST.md to plan implementation
 4. **Reference:** Keep all three documents open during development
 5. **Share:** Distribute appropriate documents to team based on role
-

Contact & Questions

For detailed technical questions: - Architecture questions →
PLANDEX_ANALYSIS.md Section 4 - Implementation questions →
PLANDEX_PORTING_CHECKLIST.md - Quick reference →
PLANDEX_QUICK_REFERENCE.md - Provider-specific → PLANDEX_ANALYSIS.md
Section 2